

That's actually a **very smart approach**—breaking Module 4 into sub-modules will make your explanation **clean, structured, and mentor-friendly**.

I'll give you the **complete blueprint (like a document outline)** so you can build your report step-by-step without missing anything.

MODULE 4 — EMAIL SENDING SYSTEM (MASTER DOCUMENT STRUCTURE)

Goal of Module 4

Build a **scalable, fault-tolerant email campaign system** that can:

- ✓ Create campaigns
 - ✓ Add recipients
 - ✓ Dispatch emails asynchronously
 - ✓ Handle failures (retry)
 - ✓ Avoid duplicates (idempotency)
 - ✓ Track events (SENT / FAILED)
-

HOW WE SPLIT INTO 5 SUB-MODULES

We divide the system logically based on **real backend architecture layers**:

● **SUB MODULE 1 — CAMPAIGN & RECIPIENT MANAGEMENT**

Covers:

- ✓ Campaign creation

- ✓ Campaign status lifecycle
- ✓ Recipient storage
- ✓ Validation rules
- ✓ Database schema (campaigns, recipients)

Purpose:

This is the **entry layer (API + DB)**

Without this:

✗ No data → nothing to send

Files Covered:

```
src/modules/campaign/*
src/modules/recipient/*
src/infrastructure/database/entities/*
```

● SUB MODULE 2 — DISPATCH & QUEUE SYSTEM

Covers:

- ✓ Dispatch API
- ✓ Queue job creation (Bull)
- ✓ Immediate vs scheduled sending
- ✓ Job structure (per recipient)
- ✓ Queue configuration (retry, backoff)

Purpose:

Convert:

Campaign → Queue Jobs

This is where system becomes **asynchronous**

Files Covered:

src/modules/dispatch/*
src/infrastructure/queue/*

● SUB MODULE 3 — WORKER PROCESSING ENGINE

✈ Covers:

- ✓ Queue worker (EmailProcessor)
- ✓ Job execution
- ✓ Idempotency (atomic DB lock)
- ✓ Retry handling
- ✓ Success/failure logic

🧠 Purpose:

Actual **email sending engine**

Without this:

- ✗ Queue exists but nothing executes

📁 Files Covered:

src/modules/worker/*

● SUB MODULE 4 — EMAIL PROVIDER & DELIVERY LAYER

✈ Covers:

- ✓ EmailService abstraction
- ✓ SES / SendGrid providers
- ✓ Dynamic provider selection
- ✓ Failure simulation (dev)
- ✓ Real provider integration design

Purpose:

This layer actually **sends emails**

Files Covered:

```
src/modules/email/*  
src/infrastructure/email/providers/*
```

● SUB MODULE 5 — EVENT TRACKING & SYSTEM FINALIZATION

Covers:

- ✓ Event storage (SENT / FAILED)
- ✓ Campaign completion logic
- ✓ Aggregation (success/failure count)
- ✓ Final campaign status update
- ✓ Webhooks (SendGrid/SES)

Purpose:

Tracks what happened after sending

Files Covered:

```
src/modules/event/*  
src/modules/webhook/*
```

COMPLETE FLOW (VERY IMPORTANT FOR MENTOR)

You must explain this clearly:

1. Create Campaign
2. Add Recipients
3. Dispatch Campaign

4. Jobs pushed to Queue (Redis)
 5. Worker processes each job
 6. Email sent via Provider
 7. Event stored
 8. Campaign marked as SENT
-

SYSTEM ARCHITECTURE (YOU CAN DRAW THIS)

Client → API → DB → Queue (Redis) → Worker → Provider → Events → Status Update

HOW TO RUN PROJECT (ADD IN DOCUMENT)

1. Start Redis (Docker)

```
docker run -d -p 6379:6379 redis
```

2. Start Backend

```
npm run start:dev
```

3. Health Check

```
Invoke-RestMethod http://localhost:3000/api/v1/health/db
```

4. Full Test Flow

Create Campaign

```
Invoke-RestMethod -Method POST http://localhost:3000/api/v1/campaigns `
  -ContentType "application/json" `
  -Body
'{"senderEmail":"test@example.com","subject":"Test","htmlContent":"<p>Hello</p>"}
```

Add Recipients

```
Invoke-RestMethod -Method POST
http://localhost:3000/api/v1/campaigns/ID/recipients `
  -ContentType "application/json" `
  -Body '{"recipients":[{"email":"a@gmail.com"}]}'
```

Dispatch

```
Invoke-RestMethod -Method POST
http://localhost:3000/api/v1/campaigns/ID/dispatch
```

KEY CONCEPTS TO EXPLAIN (MENTOR IMPRESSION)

- ✓ Asynchronous processing
 - ✓ Queue-based architecture
 - ✓ Idempotency (no duplicate emails)
 - ✓ Retry mechanism
 - ✓ Rate limiting
 - ✓ Event-driven design
-

FINAL DOCUMENT FLOW

Your report should be:

1. Introduction
 2. System Architecture
 3. Sub Module 1
 4. Sub Module 2
 5. Sub Module 3
 6. Sub Module 4
 7. Sub Module 5
 8. Testing & Results
 9. Conclusion
-

FINAL ANSWER

- ✓ YES – your plan is PERFECT
 - ✓ Splitting into 5 modules is correct
 - ✓ This matches real industry architecture
-



Say:

give sub module 1 documentation

I'll give you:

- 🔥 Full **ready-to-write document content** (mentor-level explanation)

Here is **Sub Module 1** rewritten cleanly, structured like a proper document, with **minimal relevant commands included** (only where appropriate), and easy to explain.

SUB MODULE 1

CAMPAIGN & RECIPIENT MANAGEMENT

1. Introduction

Sub Module 1 handles the **core data layer** of the Email Sending System. It is responsible for creating campaigns and managing recipients associated with those campaigns.

This module acts as the **foundation of the system**, because all further operations such as dispatching, queue processing, and email sending depend on the data created here.

2. Objectives

The objectives of this module are:

- Create email campaigns
 - Store campaign details in the database
 - Add recipients to campaigns
 - Prevent duplicate recipients
 - Validate input data
 - Maintain campaign lifecycle states
-

3. Role in System Architecture

This module operates at the **API + Database layer**.

Flow:

```
Client → Campaign API → Database
      → Recipient API → Database
```

It prepares structured data that is later used by:

- Dispatch system (Sub Module 2)
 - Queue system
 - Worker processing engine
-

4. Database Design

4.1 Campaign Entity

Stores campaign information.

Fields:

- id (UUID, Primary Key)
- senderEmail
- subject
- htmlContent
- status
- scheduleTime
- createdAt

Status Flow:

```
draft → processing → sent
```

4.2 Recipient Entity

Stores recipients for each campaign.

Fields:

- id (UUID)
- campaignId (Foreign Key)
- email
- firstName
- dispatchStatus
- createdAt

4.3 Dispatch Status

Each recipient has its own processing state:

pending → processing → sent / failed

Purpose:

- Avoid duplicate email sending (idempotency)
- Track progress
- Support retry logic

5. Code Structure

```
src/modules/campaign/  
src/modules/recipient/  
src/infrastructure/database/entities/
```

5.1 Campaign Module

Handles:

- Creating campaigns
- Fetching campaign details
- Maintaining campaign status

5.2 Recipient Module

Handles:

- Bulk insertion of recipients
- Duplicate prevention
- Linking recipients to campaigns

5.3 Entity Layer

Defines database schema using ORM (TypeORM):

- campaign.entity.ts
- recipient.entity.ts

6. API Endpoints

6.1 Create Campaign

POST /api/v1/campaigns

Request Body:

```
{
  "senderEmail": "test@example.com",
  "subject": "Test Campaign",
  "htmlContent": "<p>Hello</p>"
}
```

Response:

- Campaign ID
- Status = draft

6.2 Add Recipients

POST /api/v1/campaigns/:campaignId/recipients

Request Body:

```
{
  "recipients": [
    { "email": "a@gmail.com", "firstName": "A" },
    { "email": "b@gmail.com", "firstName": "B" }
  ]
}
```

7. Core Logic

7.1 Bulk Insert with Duplicate Protection

Recipients are inserted using a conflict-safe approach:

```
ON CONFLICT DO NOTHING
```

This ensures:

- No duplicate recipients per campaign
 - Safe repeated API calls
-

7.2 Validation Rules

- Email must be valid
 - Subject must have minimum length
 - HTML content must not be empty
 - Campaign must exist before adding recipients
-

7.3 Campaign State Control

- New campaign → draft
 - Only draft campaigns can proceed to dispatch
 - Status changes handled in later modules
-

8. Error Handling

Common errors handled:

- Invalid campaign ID
- Empty recipient list
- Invalid email format
- Duplicate entries

HTTP responses:

400 → Bad Request
404 → Not Found

9. Example Testing (Relevant to This Module)

Create Campaign

```
Invoke-RestMethod -Method POST http://localhost:3000/api/v1/campaigns `
  -ContentType "application/json" `
  -Body
'{"senderEmail":"test@example.com","subject":"Test","htmlContent":"<p>Hello</p>"}
```

Add Recipients

```
Invoke-RestMethod -Method POST
http://localhost:3000/api/v1/campaigns/ID/recipients `
  -ContentType "application/json" `
  -Body '{"recipients":[{"email":"a@gmail.com"}]}'
```

10. Importance of This Module

This module ensures:

- Structured and valid data storage
- Proper linkage between campaigns and recipients
- No duplication of recipients
- Clean input for downstream processing

Without this module:

No campaign → No recipients → No email system

11. Advantages

- Scalable database structure
 - Clean separation of responsibilities
 - Safe bulk insertion
 - Supports large recipient datasets
-

12. Limitations

- No user authentication
 - No UI/dashboard
 - No recipient segmentation or filtering
-

13. Summary

Sub Module 1 forms the **data foundation of the email system**. It ensures that campaigns and recipients are stored correctly and validated before any processing begins. All other modules depend on the correctness of this module.

NEXT

Say:

`give sub module 2 documentation`

and we'll continue in the same clean format.

Here is **Sub Module 2 – Dispatch & Queue System**, written in the same clean, professional, mentor-ready format.

SUB MODULE 2

DISPATCH & QUEUE SYSTEM

1. Introduction

Sub Module 2 is responsible for converting campaign data into **asynchronous processing jobs** using a queue system.

This module takes a campaign with recipients and pushes individual email jobs into a queue powered by Redis. It ensures that emails are not sent directly from the API, but are processed in a scalable and controlled manner.

2. Objectives

The objectives of this module are:

- Trigger email sending via dispatch API
 - Convert recipients into queue jobs
 - Enable asynchronous processing
 - Support immediate and scheduled dispatch
 - Configure retry and backoff strategies
 - Ensure safe dispatch execution
-

3. Role in System Architecture

This module acts as the **bridge between database and worker**.

Flow:

Campaign Data → Dispatch API → Queue (Redis) → Worker (next module)

It ensures that:

- API remains fast
 - Email sending is offloaded to background processing
-

4. Key Concepts

4.1 Asynchronous Processing

Instead of sending emails directly:

API → Queue → Worker → Email

This avoids:

- Blocking API requests
 - Timeout issues
 - Performance bottlenecks
-

4.2 Queue System (Bull + Redis)

- Redis acts as message broker
 - Bull manages job queue
 - Each recipient becomes a separate job
-

5. Code Structure

```
src/modules/dispatch/  
src/infrastructure/queue/
```

5.1 Dispatch Module

Handles:

- Dispatch API
 - Validations before queueing
 - Campaign status updates
-

5.2 Queue Layer

Handles:

- Job creation
 - Queue configuration
 - Retry mechanism
 - Rate limiting
-

6. API Endpoint

Dispatch Campaign

POST /api/v1/campaigns/:campaignId/dispatch

7. Dispatch Flow

When dispatch is triggered:

1. Validate campaign status (must be draft)
 2. Fetch recipients
 3. Update campaign → processing
 4. Create jobs (one per recipient)
 5. Push jobs into queue
-

8. Job Structure

Each job contains:

```
{
  campaignId,
  email,
  firstName
}
```

Each recipient is processed independently.

9. Queue Configuration

9.1 Retry Logic

Jobs are configured with retry attempts:

Attempts: multiple retries on failure

9.2 Backoff Strategy

Retry delay increases gradually

Prevents:

- Spamming provider
 - Immediate repeated failures
-

9.3 Rate Limiting

Limits number of jobs per second

Ensures:

- Controlled email sending
 - Avoid provider throttling
-

10. Campaign State Management

During dispatch:

draft → processing

This prevents:

- Duplicate dispatch calls
 - Multiple job creation
-

11. Error Handling

Handles:

- Campaign not found
- Invalid campaign status
- No recipients
- Duplicate dispatch attempts

Example:

409 → Campaign already processing/sent
400 → Invalid request

12. Example Testing (Dispatch)

```
Invoke-RestMethod -Method POST  
http://localhost:3000/api/v1/campaigns/ID/dispatch
```

Expected Response:

```
queued : True
```

13. Importance of This Module

This module ensures:

- Decoupling between API and email sending
- Scalable job handling
- Reliable background processing
- Smooth handling of large recipient lists

Without this module:

```
API would directly send emails → slow and unreliable
```

14. Advantages

- High scalability
 - Fault tolerance (retry mechanism)
 - Non-blocking API
 - Supports large campaigns
-

15. Limitations

- Requires Redis to be running
 - Slight delay due to async processing
 - Queue monitoring not implemented (UI missing)
-

16. Summary

Sub Module 2 transforms campaign data into queue jobs and enables asynchronous processing. It ensures that email sending is handled efficiently, reliably, and at scale without affecting API performance.

NEXT

Say:

give sub module 3 documentation

We'll continue with the **Worker Processing Engine (core logic of system)**.

Here is **Sub Module 3 – Worker Processing Engine**, written in the same structured, clear, mentor-ready format.

SUB MODULE 3

WORKER PROCESSING ENGINE

1. Introduction

Sub Module 3 is responsible for **processing queued jobs and executing the email sending logic**.

It acts as the **core execution engine** of the system. Once jobs are added to the queue (from Sub Module 2), this module consumes those jobs and processes each email independently.

2. Objectives

The objectives of this module are:

- Consume jobs from the queue
- Process each recipient individually
- Ensure no duplicate email is sent (idempotency)
- Handle success and failure cases
- Update recipient status in database
- Trigger event tracking

3. Role in System Architecture

This module acts as the **execution layer**.

Flow:

Queue → Worker → Email Provider → Event Tracking

It ensures that:

- Each job is processed reliably
- Failures are handled properly
- System remains consistent

4. Key Concepts

4.1 Worker (Processor)

The worker listens to the queue and processes jobs using:

```
@Processor('email')
@Process('send-email')
```

Each job represents **one email**.

4.2 Idempotency (Critical Feature)

To avoid duplicate email sending, the system uses:

```
dispatch_status check in database
```

Flow:

pending → processing → sent

If a job is retried or duplicated:

- Already processed emails are skipped
-

4.3 Atomic Update (Safe Processing)

Before sending email:

```
UPDATE recipient SET status = processing WHERE status = pending
```

If update fails:

- Another worker already processed it
 - Job is skipped
-

5. Code Structure

```
src/modules/worker/
```

5.1 Email Processor

Main class responsible for:

- Receiving jobs
 - Executing email logic
 - Updating database
 - Logging results
-

6. Job Processing Flow

For each job:

1. Receive job from queue
 2. Check recipient status
 3. Lock row (processing)
 4. Send email
 5. Update status (sent/failed)
 6. Store event
 7. Continue to next job
-

7. Success Handling

If email is sent successfully:

```
dispatchStatus → sent  
Event → SENT stored
```

8. Failure Handling

If email fails:

```
dispatchStatus → failed  
Event → FAILED stored
```

Retry logic (from queue) will:

- Attempt again automatically
-

9. Logging

Worker logs include:

```
Sending email → user@example.com  
Email sent successfully  
Skipping duplicate email
```

These logs help in:

- Debugging
 - Monitoring
 - System verification
-

10. Example Execution (What Mentor Will See)

During dispatch:

```
Sending email → a@gmail.com  
Sending email → b@gmail.com  
Sending email → c@gmail.com
```

This confirms:

- Worker is processing jobs correctly
-

11. Integration with Other Modules

- Receives jobs from **Dispatch Module**
 - Calls **Email Provider (Sub Module 4)**
 - Sends results to **Event Module (Sub Module 5)**
-

12. Importance of This Module

This module is the **heart of the system**.

Without it:

Jobs exist → but nothing executes

13. Advantages

- Independent job execution
 - Fault-tolerant (retry support)
 - Prevents duplicate emails
 - Scales with queue workers
-

14. Limitations

- Requires Redis + queue setup
 - No horizontal scaling implemented yet (single worker)
 - No monitoring dashboard
-

15. Summary

Sub Module 3 processes each queued job and executes the actual email sending logic. It ensures reliability, prevents duplicates, and maintains data consistency using idempotency and atomic operations.

NEXT

Say:

give sub module 4 documentation

Next is **Email Provider Layer (actual sending abstraction)**.

Here is **Sub Module 4 – Email Provider & Delivery Layer**, written in the same clear, structured, mentor-ready format.

SUB MODULE 4

EMAIL PROVIDER & DELIVERY LAYER

1. Introduction

Sub Module 4 is responsible for the **actual email delivery logic**. It abstracts the communication with external email providers such as AWS SES or SendGrid.

This module ensures that the system is **provider-independent**, meaning the underlying email service can be changed without affecting other parts of the system.

2. Objectives

The objectives of this module are:

- Send emails using external providers
- Provide a common interface for email sending
- Support multiple providers (SES, SendGrid)
- Enable easy switching between providers
- Handle provider failures gracefully
- Support development (mock/simulated sending)

3. Role in System Architecture

This module acts as the **delivery layer**.

Flow:

Worker → Email Service → Provider (SES/SendGrid) → External Email System

It is invoked by:

- Worker (Sub Module 3)
-

4. Key Concept: Abstraction Layer

Instead of directly calling a provider, the system uses:

EmailService (common interface)

This allows:

Switch provider without changing business logic

5. Code Structure

```
src/modules/email/  
src/infrastructure/email/providers/
```

5.1 Email Service

Acts as a central interface:

- Receives email request
 - Chooses provider
 - Calls appropriate provider implementation
-

5.2 Providers

Examples:

- ses.provider.ts
- sendgrid.provider.ts

Each provider implements:

- `sendEmail()` method
-

6. Email Sending Flow

1. Worker calls `EmailService`
 2. `EmailService` selects provider
 3. Provider sends email
 4. Response returned to worker
-

7. Development Mode (Important)

In development, if API keys are not provided:

Emails are simulated instead of actually sent

Example log:

```
DEV MODE: Simulating email send
```

Purpose:

- Safe testing
 - No real emails sent
-

8. Production Mode

When configured properly:

- Real API keys are used
 - Emails are sent via provider
 - Delivery is handled externally
-

9. Error Handling

Handles:

- Invalid API keys
- Provider failure

- Network issues

In case of failure:

- Error is returned to worker
 - Worker marks email as failed
 - Retry mechanism is triggered
-

10. Provider Flexibility

The system supports:

Single provider → default

Multiple providers → switch logic possible

Future enhancement:

- Load balancing between providers
 - Failover mechanism
-

11. Example Behavior

Development Mode

Sending email → a@gmail.com

DEV MODE: Simulated send

Production Mode

Sending email → a@gmail.com

Email sent via SendGrid

12. Integration with Other Modules

- Called by **Worker Module**
 - Returns results for **Event Tracking Module**
-

13. Importance of This Module

This module ensures:

- Clean separation between logic and external services
- Flexibility in choosing email providers
- Safe testing environment

Without this module:

System becomes tightly coupled to one provider

14. Advantages

- Provider independence
 - Easy integration of new providers
 - Clean architecture
 - Safe development mode
-

15. Limitations

- No real provider configured by default
 - No provider failover implemented
 - No delivery analytics integration
-

16. Summary

Sub Module 4 abstracts the email sending process and enables communication with external email providers. It ensures flexibility, modularity, and safe testing by separating delivery logic from core business logic.

NEXT

Say:

give sub module 5 documentation

Final module: **Event Tracking & System Finalization (completion logic)**

Here is **Sub Module 5 – Event Tracking & System Finalization**, written in the same clean, structured, mentor-ready format.

SUB MODULE 5

EVENT TRACKING & SYSTEM FINALIZATION

1. Introduction

Sub Module 5 is responsible for tracking the outcome of each email and determining the **final state of a campaign**.

It records whether each email was successfully sent or failed, and then aggregates these results to update the overall campaign status.

2. Objectives

The objectives of this module are:

- Record email delivery events (SENT / FAILED)
 - Track per-recipient outcomes
 - Aggregate results for each campaign
 - Determine campaign completion
 - Update final campaign status
 - Support future webhook integrations
-

3. Role in System Architecture

This module acts as the **final stage of the pipeline**.

Flow:

Worker → Event Service → Database → Campaign Status Update

It ensures that:

- Every email result is recorded
 - Campaign completion is accurately determined
-

4. Key Concepts

4.1 Event-Based Tracking

Each email generates an event:

SENT or FAILED

These events are stored in the database for:

- Tracking
 - Analytics
 - Debugging
-

4.2 Aggregation Logic

After processing emails, the system calculates:

Total sent + total failed = total processed

When all recipients are processed:

Campaign → marked as SENT (completed)

5. Code Structure

```
src/modules/event/  
src/modules/webhook/
```

5.1 Event Module

Handles:

- Recording events
 - Counting results
 - Aggregation logic
-

5.2 Webhook Module (Optional)

Handles:

- External provider callbacks (e.g., SendGrid, SES)
 - Delivery confirmations
-

6. Event Storage

Each event contains:

```
campaignId
recipient email
eventType (SENT / FAILED)
timestamp
```

7. Processing Flow

For each email:

1. Worker processes email
 2. Result generated (success/failure)
 3. Event stored in database
 4. Aggregation check triggered
-

8. Campaign Completion Logic

The system checks:

If all recipients are processed

Then:

Update campaign status → sent

9. Example Logs

```
Stored event [SENT] for a@gmail.com  
Stored event [FAILED] for b@gmail.com  
campaign completed success=2 failed=1  
Campaign fully sent
```

10. Integration with Other Modules

- Receives data from **Worker Module**
 - Uses database entities from **Sub Module 1**
 - Finalizes campaign started in **Dispatch Module**
-

11. Error Handling

Handles:

- Event storage failure
- Inconsistent counts
- Partial processing cases

Ensures:

- System does not crash
 - State remains consistent
-

12. Importance of This Module

This module ensures:

- Complete visibility of system behavior
- Accurate campaign results
- Reliable tracking of success/failure

Without this module:

No tracking → No confirmation → No final status

13. Advantages

- Clear tracking of every email
 - Reliable campaign completion detection
 - Supports analytics and reporting
 - Enables future integrations
-

14. Limitations

- No UI dashboard for events
 - Limited analytics (basic counts only)
 - Webhook integration may be partial
-

15. Summary

Sub Module 5 records the outcome of each email and determines when a campaign is complete. It ensures that the system provides accurate results and maintains consistency across all operations.

FINAL NEXT STEP

Now your full Module 4 documentation is complete.

If you want, next I can give:

1. Full Execution & Testing Section (commands + explanation)
2. System Architecture diagram explanation
3. Viva/mentor Q&A preparation
4. Resume-ready project description

Just tell me what you want next.

Here is the **Execution & Testing Section** for your document. This is written in a clear, step-by-step format so you can both **run the system** and **explain it to your mentor confidently**.

EXECUTION & TESTING OF MODULE 4

EMAIL SENDING SYSTEM

1. Prerequisites

Before running the system, ensure the following are available:

- Node.js installed
 - npm installed
 - Docker installed and running
 - Project dependencies installed (`node_modules`)
-

2. Install Dependencies

Navigate to the project folder and run:

```
npm install
```

3. Start Redis (Queue System)

The system uses Redis for queue processing.

Run:

```
docker run -d -p 6379:6379 redis
```

Important Note:

If port is already in use:

```
Redis is already running → no need to start again
```

To verify:

```
docker ps
```

4. Start Backend Server

Run:

```
npm run start:dev
```

Expected Output:

```
Nest application successfully started  
Email Sending Engine running on port 3000
```

5. Health Check

Verify server and database connection:

```
Invoke-RestMethod http://localhost:3000/api/v1/health/db
```

Expected Output:

```
status : ok
```

6. Testing Flow (Step-by-Step)

This demonstrates the complete working of the system.

6.1 Create Campaign

```
Invoke-RestMethod -Method POST http://localhost:3000/api/v1/campaigns `
  -ContentType "application/json" `
  -Body '{"senderEmail":"test@example.com","subject":"Test  
Campaign","htmlContent":"<p>Hello {{firstName}}</p>"}'
```

Expected Output:

```
status : draft  
id : <campaign_id>
```

Save the campaign ID for next steps.

6.2 Add Recipients

```
Invoke-RestMethod -Method POST  
http://localhost:3000/api/v1/campaigns/ID/recipients `
  -ContentType "application/json" `
  -Body '{
```

```
"recipients":[
  {"email":"a@gmail.com","firstName":"A"},
  {"email":"b@gmail.com","firstName":"B"},
  {"email":"c@gmail.com","firstName":"C"}
]
```

Expected Output:

```
stored : 3
```

6.3 Dispatch Campaign

```
Invoke-RestMethod -Method POST
http://localhost:3000/api/v1/campaigns/ID/dispatch
```

Expected Output:

```
queued : True
```

7. Observe Worker Processing (Very Important)

After dispatch, check the backend terminal.

Expected Logs:

```
Sending email → a@gmail.com
Sending email → b@gmail.com
Sending email → c@gmail.com
```

In development mode:

```
DEV MODE: Simulating email send
```

8. Verify Campaign Status

```
Invoke-RestMethod http://localhost:3000/api/v1/campaigns/ID
```

Expected Output:

```
status : sent
```

9. Duplicate Dispatch Test

Run dispatch again:

```
Invoke-RestMethod -Method POST  
http://localhost:3000/api/v1/campaigns/ID/dispatch
```

Expected Output:

```
409 CONFLICT  
Campaign already processed
```

This confirms:

- Idempotency is working
- Duplicate emails are prevented

10. Failure & Retry Behavior

If email sending fails:

- Status becomes `failed`
- Retry mechanism triggers automatically
- Event is recorded

Logs will show retry attempts.

11. Complete Flow Summary

1. Campaign created → stored in DB
2. Recipients added → stored in DB
3. Dispatch triggered → jobs added to queue
4. Worker processes each job
5. Email sent (or simulated)
6. Event stored (SENT / FAILED)
7. Campaign marked as SENT

12. Key Observations for Mentor

You can explain:

- System uses **asynchronous queue (Redis + Bull)**
- Each recipient is processed independently

- Worker ensures **fault tolerance and retry**
 - Idempotency prevents duplicate sending
 - Event system tracks delivery results
-

13. Common Issues & Fixes

Issue: Port already in use

Fix: Kill existing process or reuse running service

Issue: Redis not running

Fix: Start Redis using Docker

Issue: 500 error on dispatch

Fix: Ensure Redis is running before starting backend

14. Conclusion

This section demonstrates that the system is:

- Fully functional
 - Scalable
 - Fault-tolerant
 - Ready for real-world usage with minor enhancements
-

NEXT (OPTIONAL)

If you want, I can now give:

- System Architecture explanation (for diagrams)
- Viva questions and answers
- Resume-ready project description

Just tell me what you need next.

